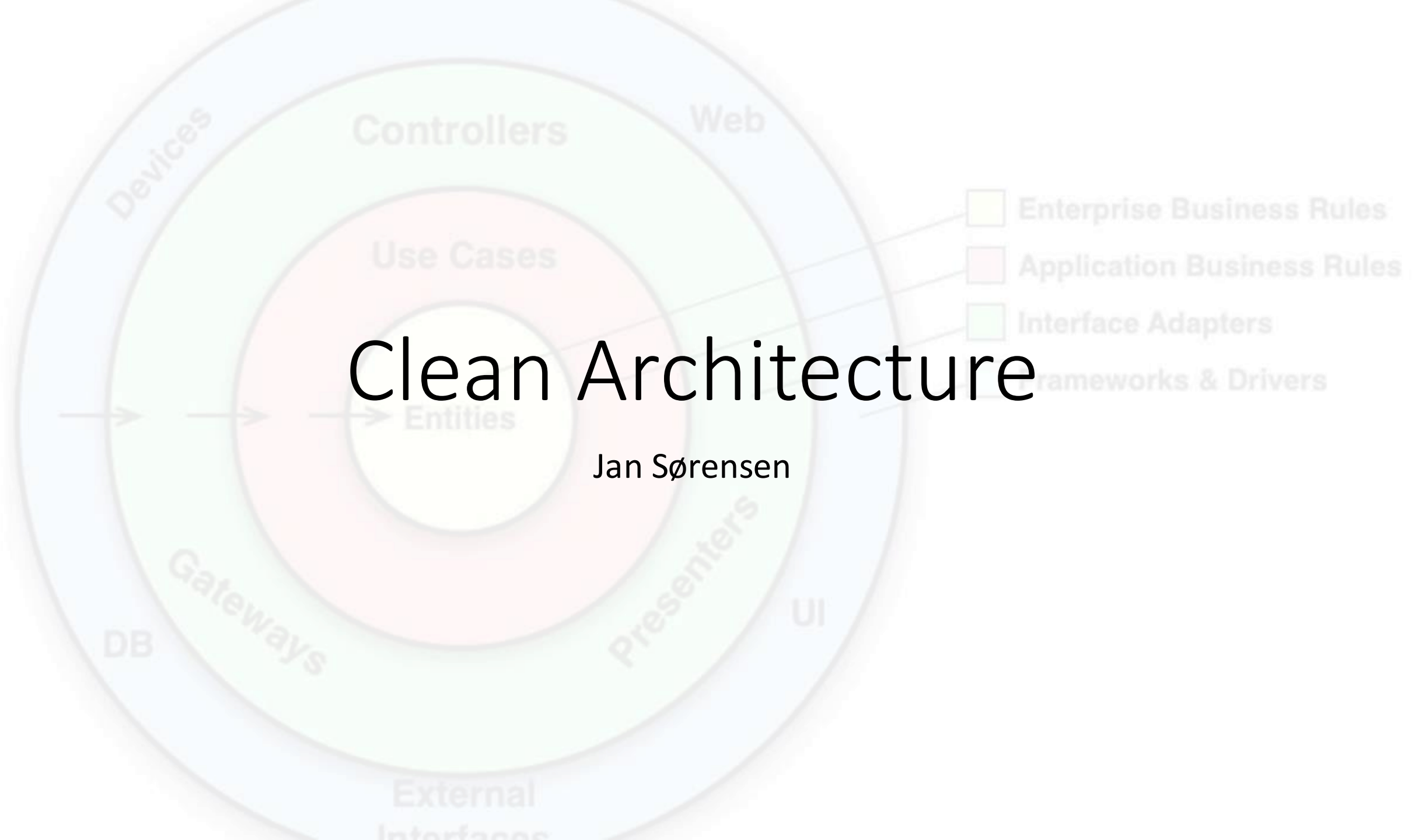


Clean Architecture

Jan Sørensen



Design pattern

A design pattern is the re-usable form of a solution to a design problem. (Singleton - Observer - MVC - MVVM - MVP)

Software architecture

Software architecture refers to the high level structures of a software system and the discipline of creating such structures and systems. (Hexagonal architecture - Onion architecture - The Clean architecture)



Design
Patter VS
Architecture

Successful SW Architecture

- Testable
- Independent of the UI
- Independent of the Database
- Independent of Frameworks and External Entities



Characteristics

Independent of frameworks.

Libraries and frameworks should use as tools, rather than having

Testable.

Business rules have tests that are independent of UI, Database

Independent of UI.

UI can change easily, without changing other system components

Independent of Database.

The database can be switched from RDBS to NoSQL or any other

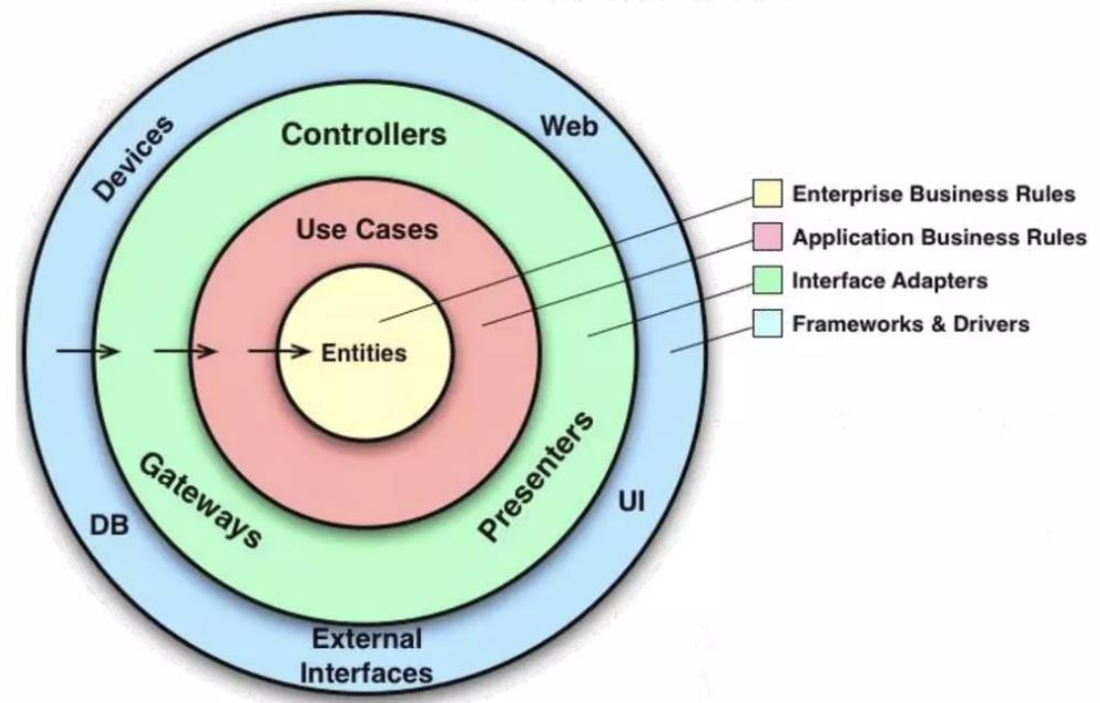
Independent of any external agency.

Business rules should know nothing about the outside world.

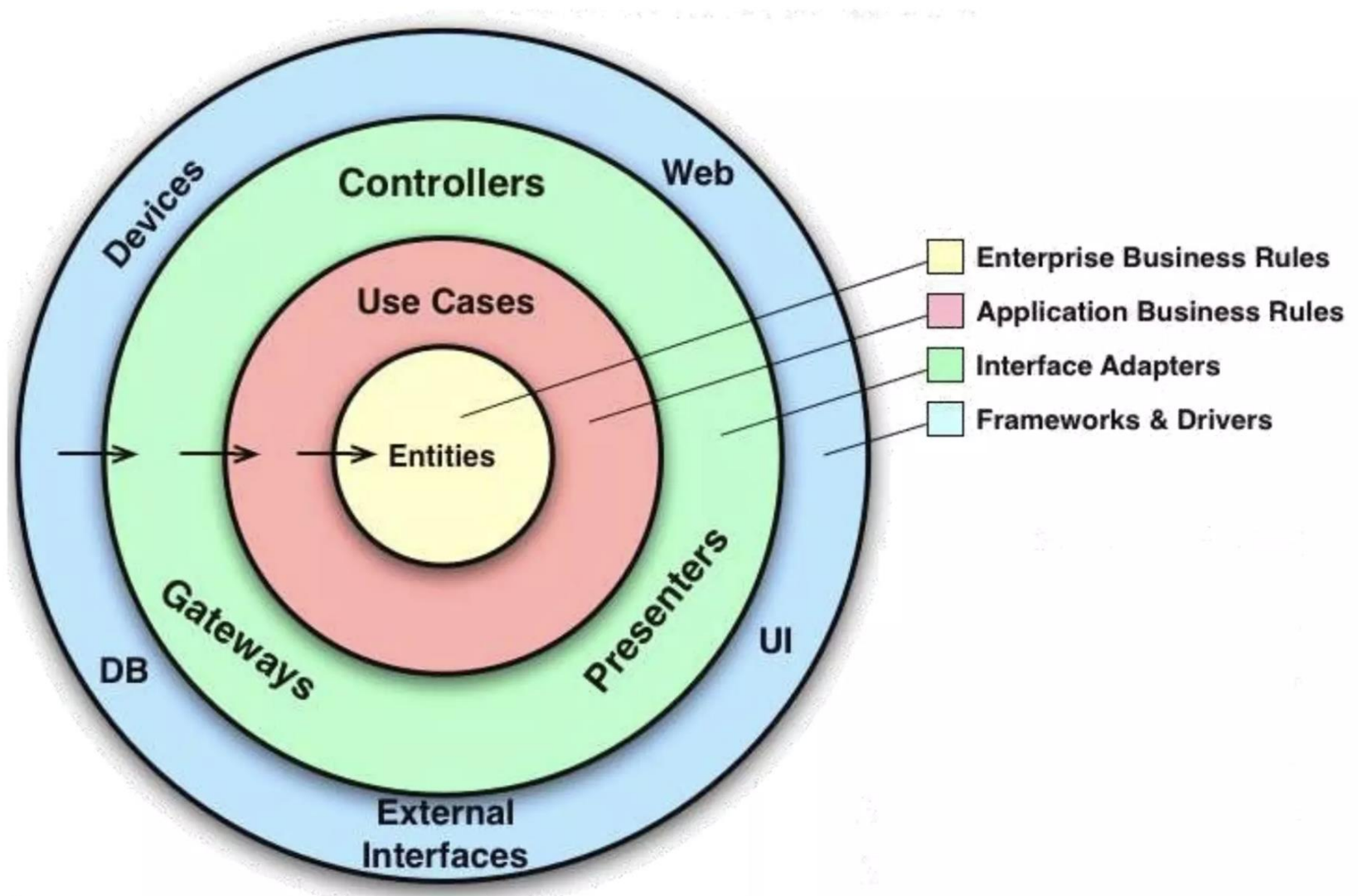
The Clean Architecture



Robert C. Martin (Clean Code)



The Clean Architecture



The Clean Architecture

Entities are *enterprise-wide* business rules that encapsulate the most general business rules, these rules are the least likely to change.

Use cases are also called interactors and stand for *application-specific* business rules of the software. This layer is isolated from changes to the database, common frameworks, and the UI.

Interface adapters convert data from a convenient format for entities and use cases to a format applicable to databases and the web, for example. This layer includes Presenters from MVP, ViewModel from MVVM, and Gateways (also known as Repositories).

Frameworks and drivers are the outermost layer, which consists of the web framework, database, UI, HTTP client, and so on.



The Clean Architecture

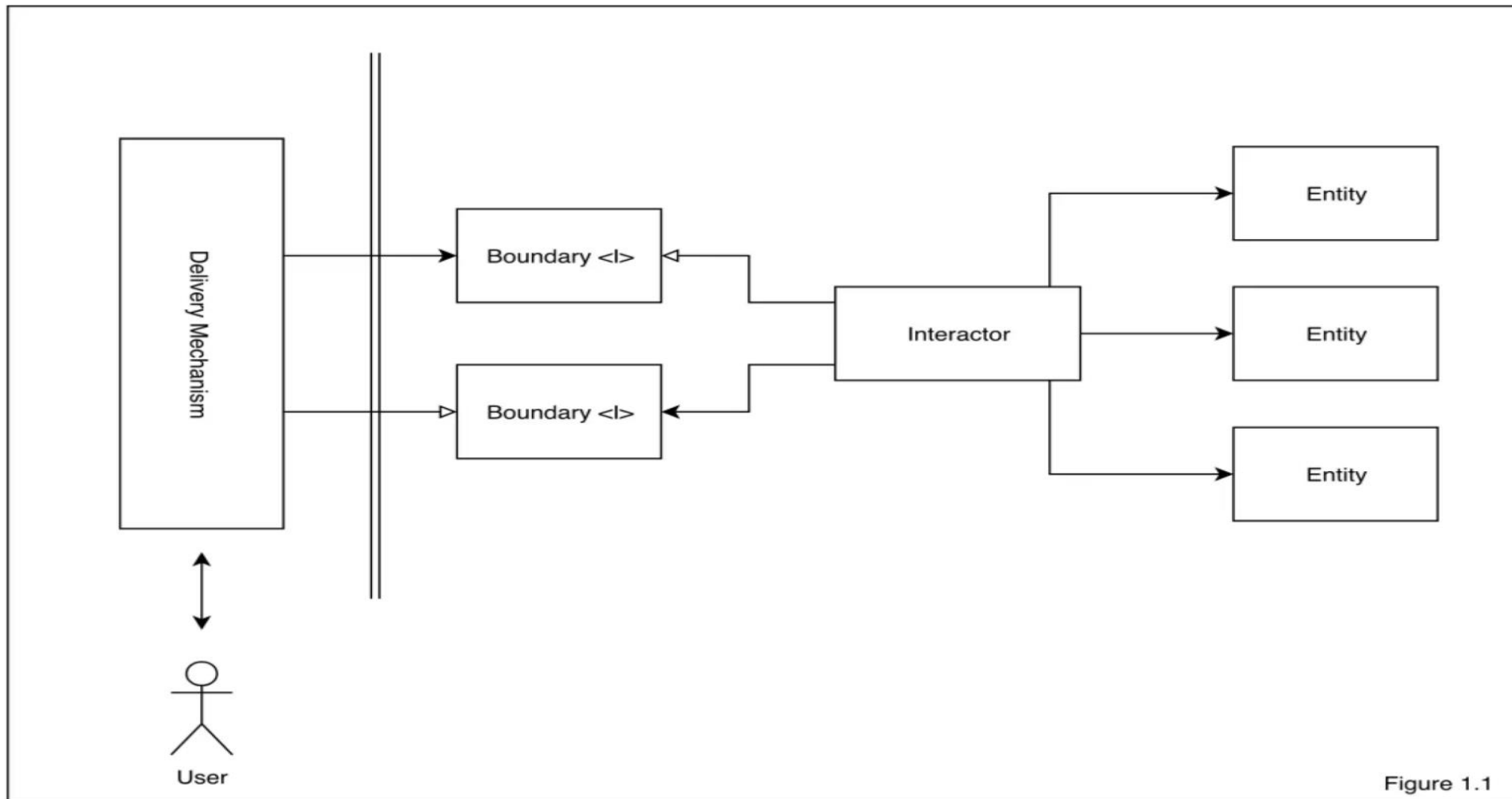


Figure 1.1

- **Interactor (Use Case):** Contains application specific business rules.
- **Entity:** Contains application independent business rules.
- **Boundary<I>:** Data comes into interactors through boundary (interface/protocol) which is implemented by the interactor. Data goes out from the interactor to boundaries implemented by other objects.
- **Delivery Mechanism:** Can be Web, Console application, Mobile App UI, etc

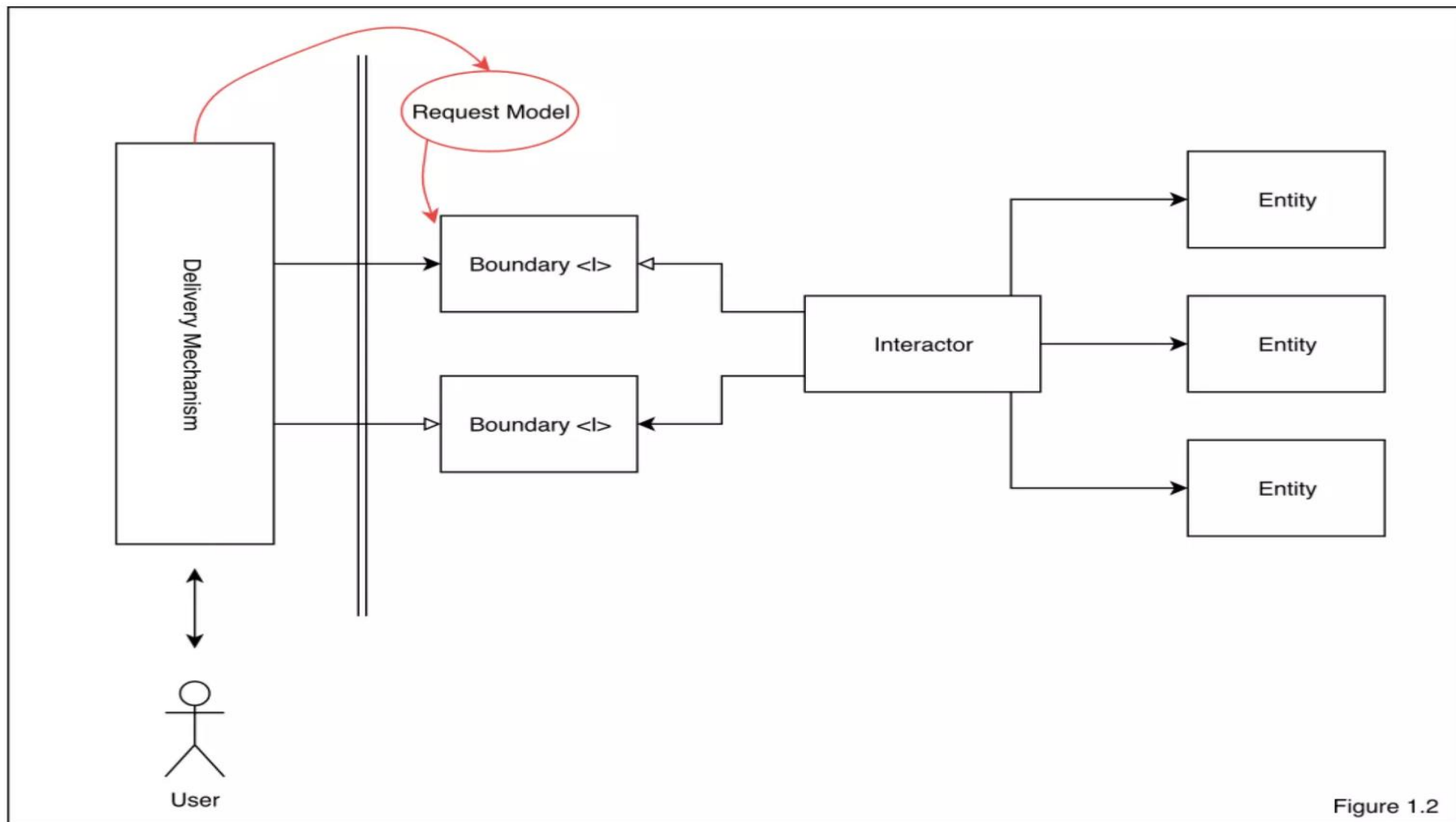
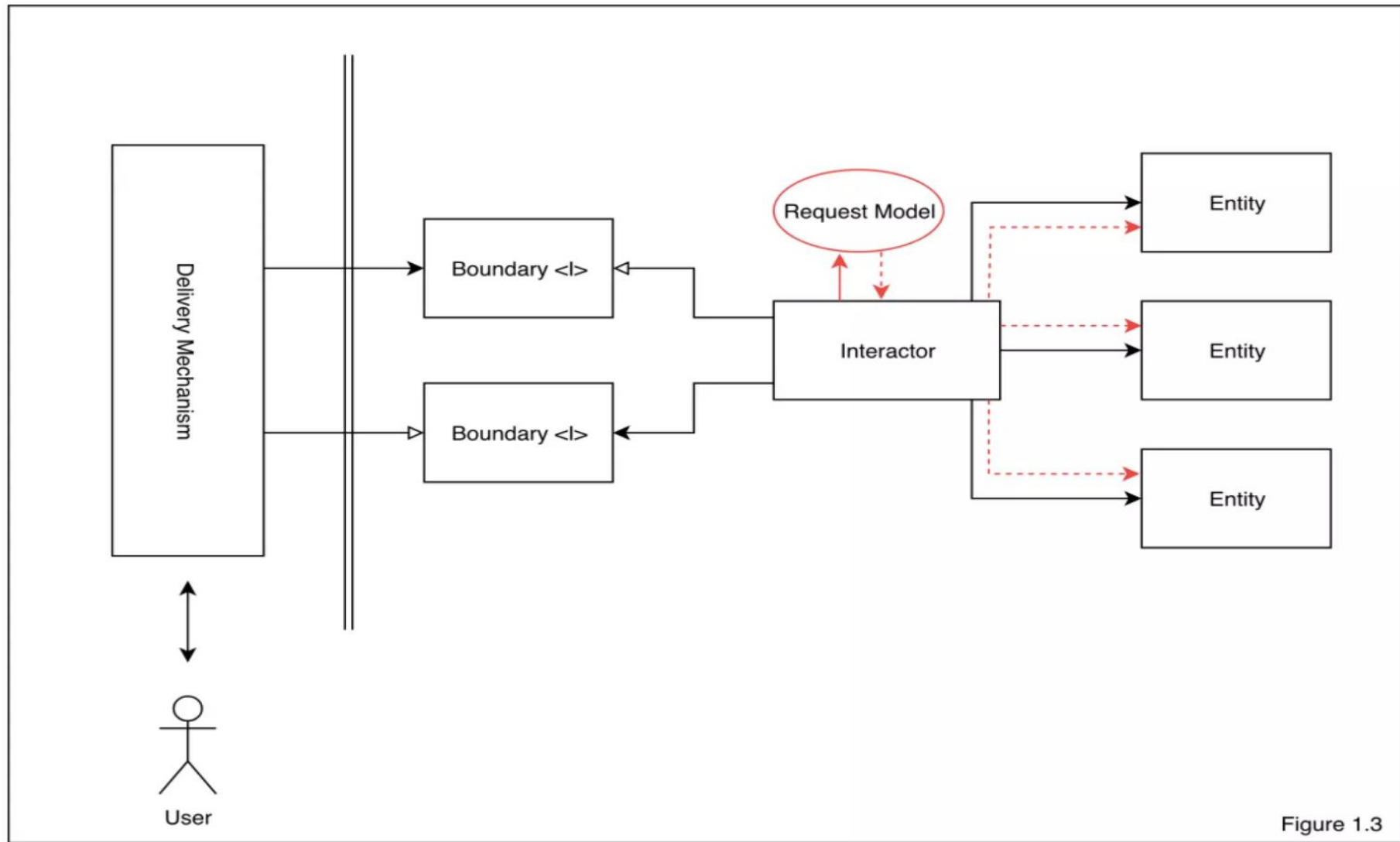
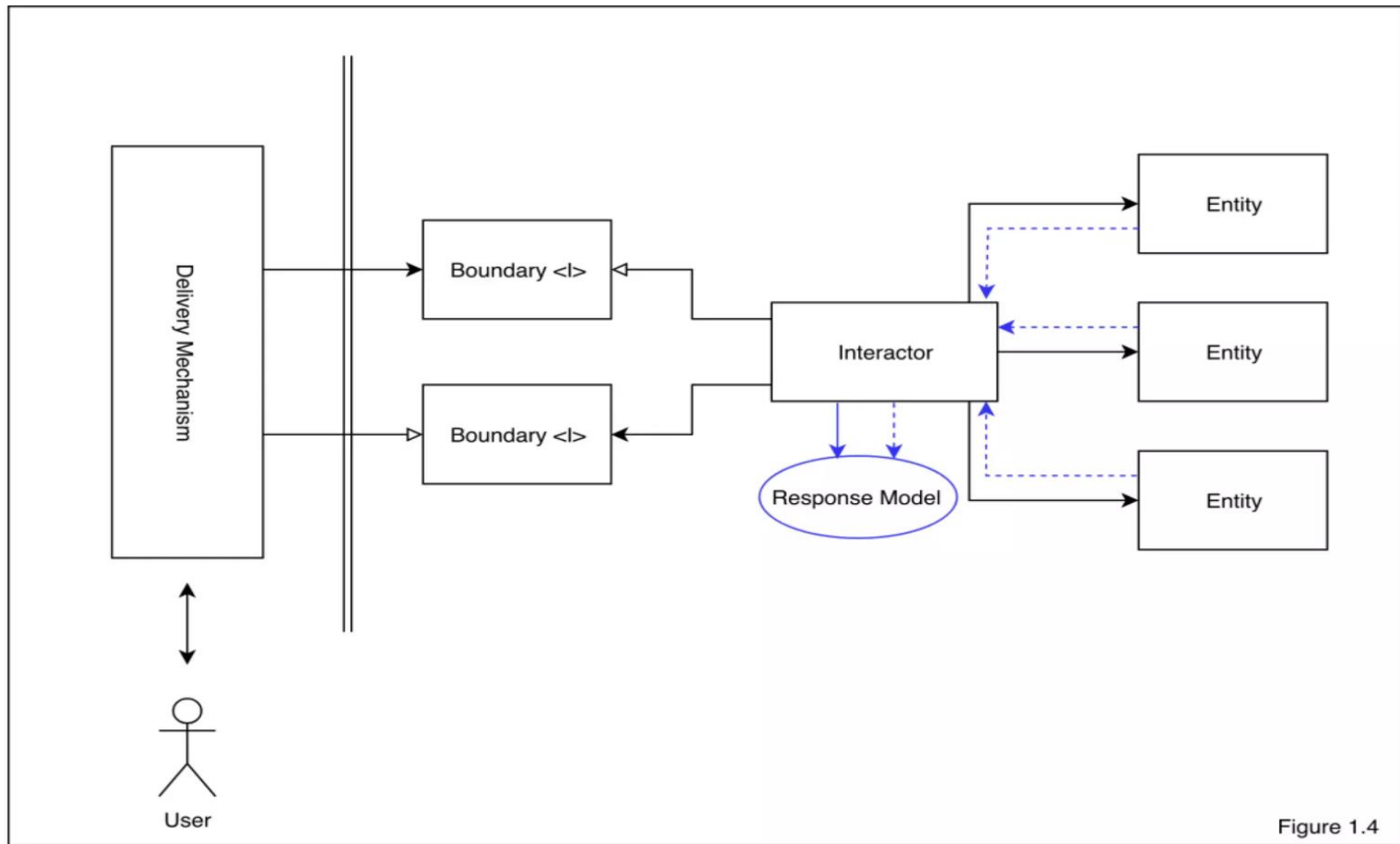


Figure 1.2

- 1- User clicks a button (on a web form for example).
- 2- The delivery mechanism (ex: Web) takes whatever data submitted by the user and stick it into a data structure which contains only primitives types (Request Model).
- 3- The request model passed through the input boundary and since the interactor implements the input boundary, the interactor received that request model.



4- Interactor then uses that request model and controls the dance of the entities (ex: creates the order, modify the customer, etc).



5- And when it's done with that, it continues to control the dance of the entities but this time in the opposite direction as it gather up all the results to create yet another data structure called (Response Model).

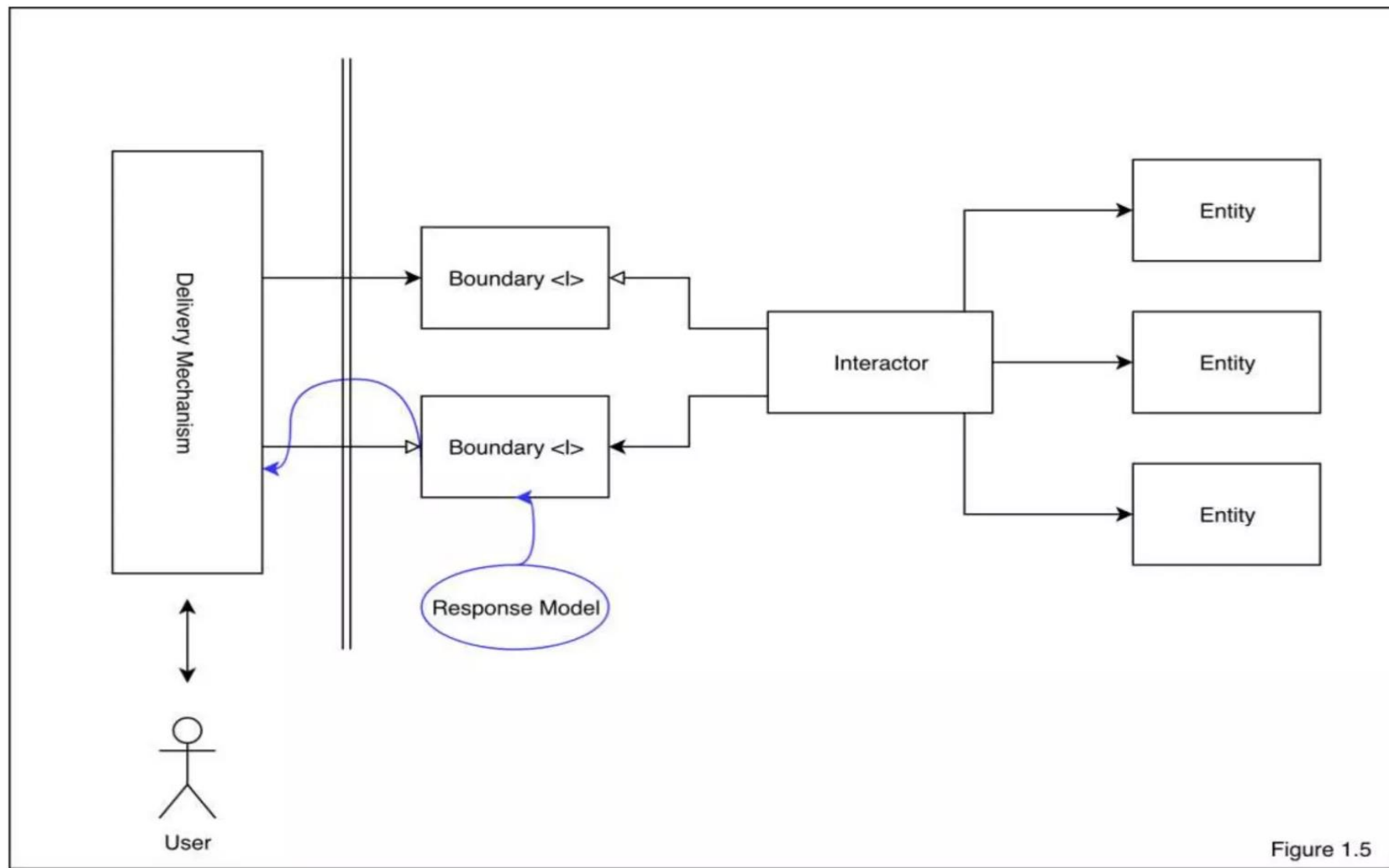


Figure 1.5

6- The response model is passed back out through the output boundary to the delivery mechanism that implement it and somehow it's delivered to the user.

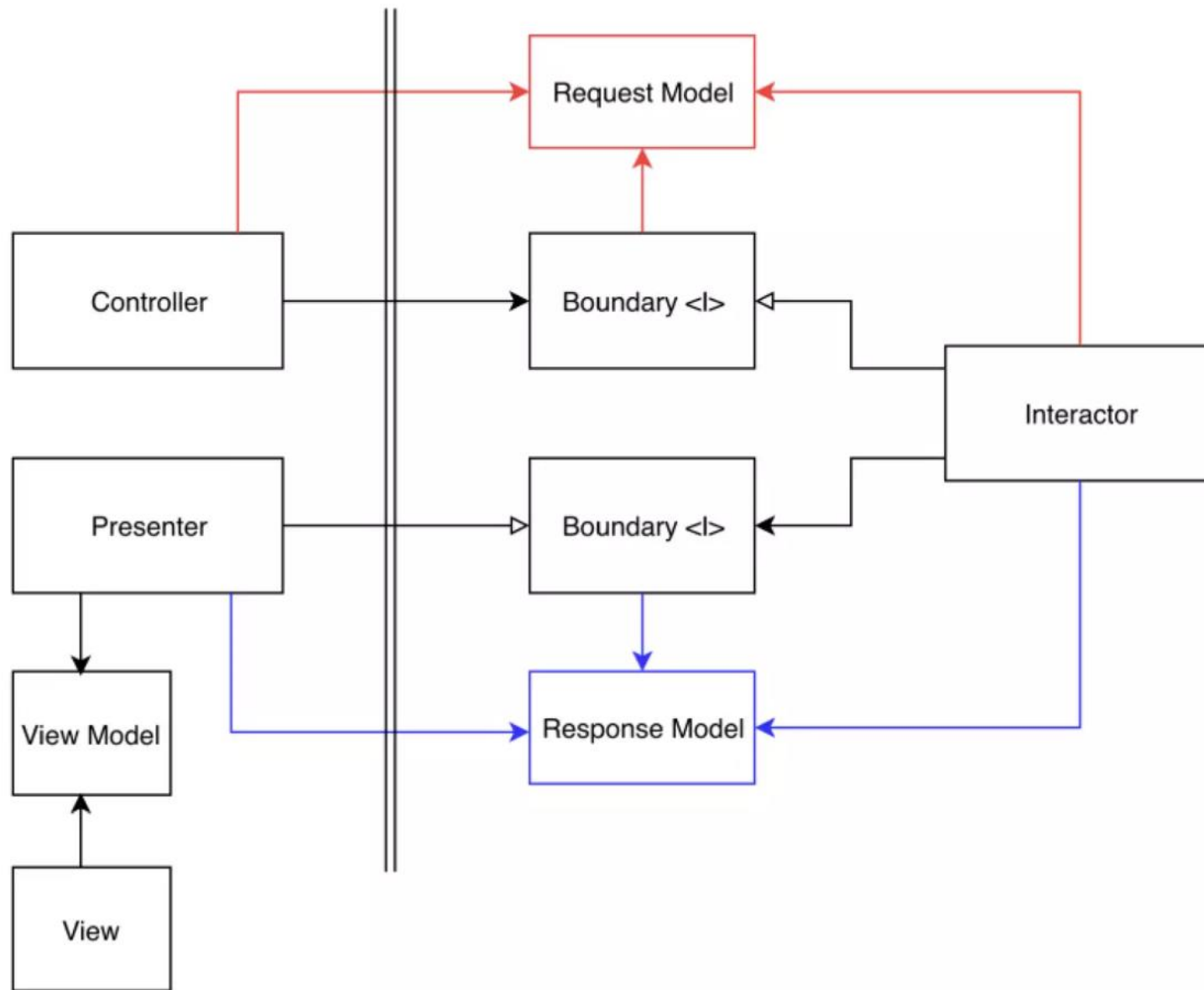


Figure 1.6

- **Response Model** contains data that was created by the interactor as the result of the use case, but this data is not presentable, a date for example would be a date object, and the currency would be a money object.
- **Presenter** is a class or a set of classes whose job is to take the response model and translate it into a view model. For example, it takes the date object from the response model and calls simple data formatter which generates a nice date string that the view model will hold, and takes the currency from the response model and puts the currency symbol in the view model.
- **View Model** contains the data that will be used by the view. For example, If there is a button on the view, the view model has the string that holds the name of that button, and if the button shouldn't be active, so it should be grey, the view model holds the boolean that indicates the status of the button. If there are menu items in the view, the view model has the names of all the menu items and their order.
- **View** all it can do is grabbing the data from the view model and put it on the screen. It should be so stupid so you don't have to test it. (You can test it with your eyes).

What about the Database

“If something changes a lot, it should be a plug-in. If something doesn’t change very often, it should be plugged into.”

—Robert C. Martin

What about the Database?

- The database is a detail, it shouldn't be the center of your architecture.
- The database should be a plug-in to the business rules.
- Business rules shouldn't be written in stored procedures in the database.

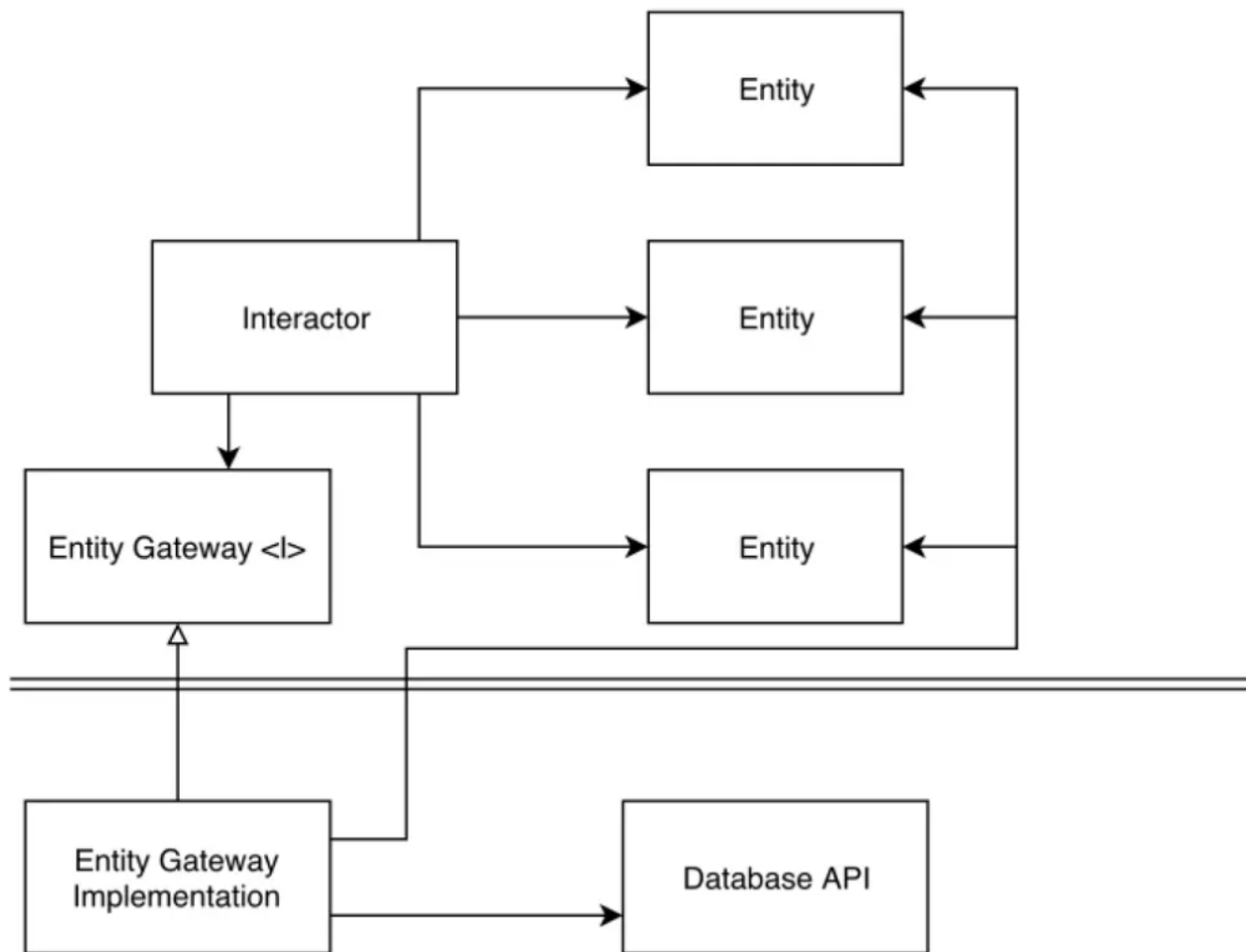


Figure 1.7

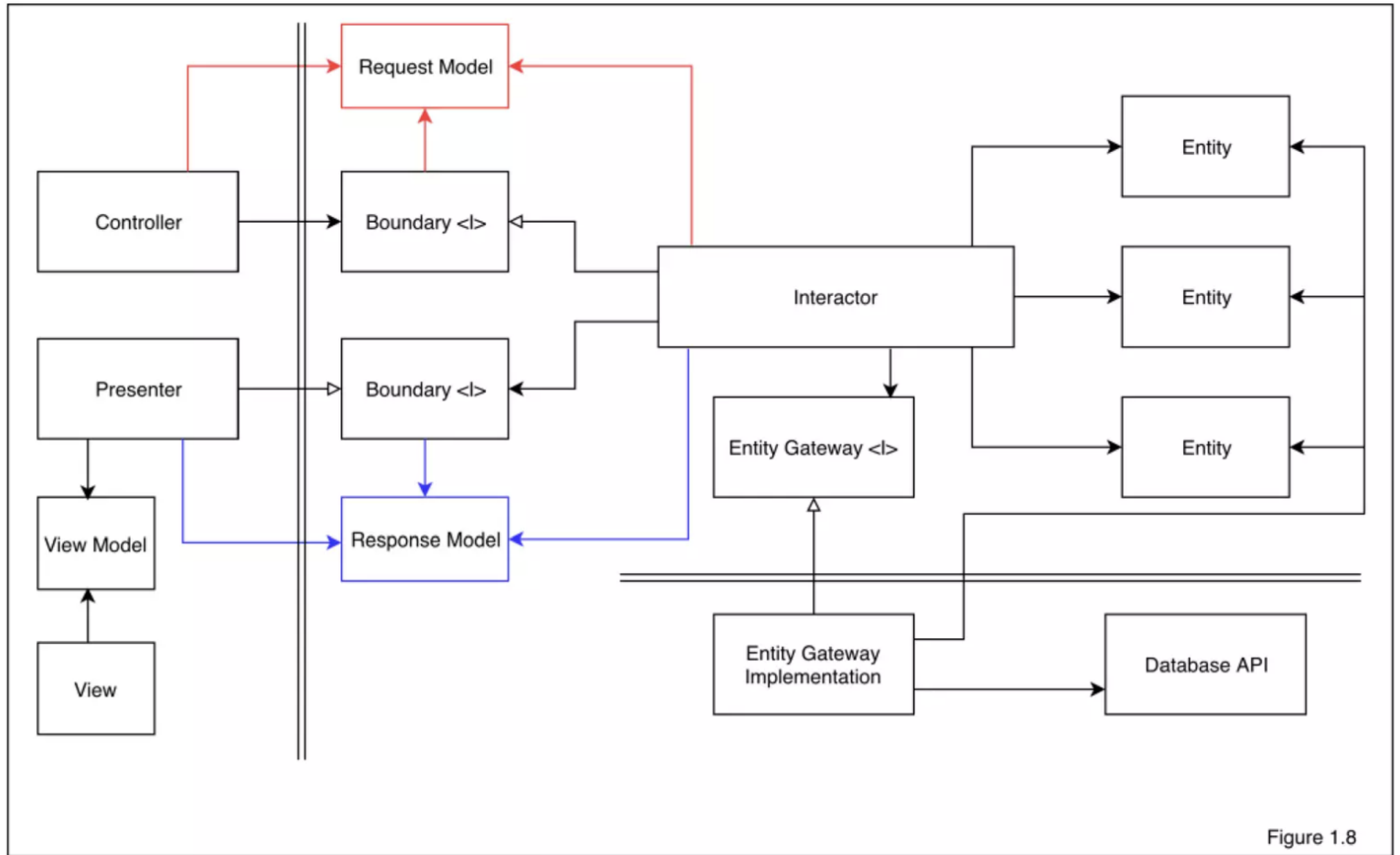


Figure 1.8

References

- https://www.youtube.com/watch?v=o_TH-Y78tt4
- <https://proandroiddev.com/clean-architecture-data-flow-dependency-rule-615ffdd79e29>
- <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
- <https://medium.freecodecamp.org/a-quick-introduction-to-clean-architecture-990c014448d2>
- <https://medium.com/@pxpgraphics/clean-architecture-3fe6907e7441>
- <https://hackernoon.com/golang-clean-architecture-efd6d7c43047>
- <https://rubygarage.org/blog/clean-android-architecture>